

LAPORAN TUGAS BESAR KLASIFIKASI WEIGHT ADJUSTED K-NEAREST NEIGHBOR (WAKNN)

Tugas Pembelajaran Mesin



Oleh:

Dimas Perkasa Agung Putra	10123133
Muhammad Ikhsan Fadillah	10123134
Rizqi Akbar Fadilah	10123151
Farhan Nawwafal Pramudia	10123470

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK DAN ILMU KOMPUTER
UNIVERSITAS KOMPUTER INDONESIA
BANDUNG

2026

1. Studi Literatur

Algoritma *K-Nearest Neighbor* (KNN) merupakan salah satu metode klasifikasi *non-parametrik* yang paling mendasar dan efektif dalam *machine learning*. Metode ini pertama kali diperkenalkan oleh Fix dan Hodges pada tahun 1951 sebagai pendekatan berbasis instans (*instance-based learning*). Prinsip dasar KNN sangat intuitif, yaitu mengklasifikasikan data baru berdasarkan mayoritas label dari K tetangga terdekatnya. Namun, meskipun sederhana dan mudah diimplementasikan, KNN standar memiliki kelemahan fundamental: algoritma ini mengasumsikan bahwa setiap fitur atau atribut dalam dataset memiliki tingkat kepentingan yang sama (*equal importance*) dalam penentuan kelas. Asumsi ini sering kali tidak valid dalam skenario dunia nyata, terutama pada dataset berdimensi tinggi di mana banyak fitur mungkin tidak relevan (*irrelevant*) atau mengandung *noise* (gangguan) yang justru mendegradasi akurasi klasifikasi.

Untuk mengatasi keterbatasan tersebut, konsep pembobotan fitur diperkenalkan melalui algoritma *Weight Adjusted K-Nearest Neighbor* (WAKNN). Variasi ini secara spesifik dipopulerkan melalui penelitian Han, Karypis, dan Kumar (2001), yang mengajukan pendekatan sistematis untuk memberikan bobot berbeda pada setiap fitur. Dalam kerangka kerja WAKNN, ruang fitur tidak lagi diperlakukan sebagai ruang Euclidean isotropik di mana setiap dimensi memiliki kontribusi seragam. Sebaliknya, WAKNN mengubah geometri ruang pencarian dengan meregangkan atau menyusutkan sumbu-sumbu fitur berdasarkan relevansinya terhadap target klasifikasi. Fitur yang memiliki daya diskriminatif tinggi akan diberikan bobot yang besar, sehingga jarak pada dimensi tersebut menjadi lebih signifikan dalam perhitungan tetangga terdekat.

Secara teknis, perbedaan utama antara WAKNN dan KNN klasik terletak pada fungsi jarak yang digunakan. Jika KNN standar umumnya menggunakan jarak Euclidean biasa, WAKNN mengadopsi *Weighted Euclidean Distance*. Proses penentuan bobot ini bukanlah sesuatu yang ditentukan secara manual, melainkan dipelajari secara otomatis dari data pelatihan. Han et al. (2001) mengusulkan penggunaan algoritma optimasi *Greedy Hill-Climbing* untuk mempelajari bobot tersebut. Metode ini bekerja secara iteratif dengan mengevaluasi dampak perubahan bobot setiap fitur terhadap kinerja klasifikasi. Algoritma akan terus menyesuaikan bobot—meningkatkan bobot fitur yang membantu prediksi yang benar dan mengurangi bobot fitur yang menyebabkan kesalahan—hingga mencapai konvergensi atau titik akurasi maksimal.

Implikasi dari penerapan WAKNN sangat signifikan, terutama dalam menangani masalah "kutukan dimensi" (*curse of dimensionality*). Dengan mekanisme pembobotan ini, fitur yang tidak relevan atau redundan secara efektif

akan mendapatkan bobot mendekati nol, sehingga pengaruh buruknya terhadap perhitungan jarak dapat diminimalisir atau dihilangkan sama sekali. Hasilnya, WAKNN tidak hanya mampu meningkatkan akurasi klasifikasi secara substansial dibandingkan KNN standar, tetapi juga memberikan wawasan tambahan mengenai fitur mana yang paling berkontribusi dalam membedakan antar kelas dalam dataset.

Berikut formula yang disajikan pada algoritma WAKNN:

1. Jarak Euclidean Weight:

$$d(x, y) = \sqrt{\sum_{i=1}^n w_i (x_i - y_i)^2}$$

2. Fungsi Similaritas:

$$Sim(X, D_j) = \frac{\sum (t \in X \cap D_j) w_t \times x_t \times d_{j,t}}{\sqrt{\sum (t \in X) (w_t \times x_t)^2} \times \sqrt{\sum (t \in D_j) (w_t \times d_{j,t})^2}}$$

3. Skor Klasifikasi:

$$Score(X, C_i) = \sum_{d_j \in KNN \cap C_i} Sim(X, D_j)$$

No	Penulis	Tahun	Data	Preprocessing	Ringkasan
1	Zheng et al.	2020	Indoor Localization (RSS signals)	Signal Filtering, Normalization	Mengembangkan WAKNN untuk lokalisasi dalam ruangan dengan menggabungkan jarak spasial dan fisik sinyal RSS untuk meningkatkan akurasi posisi hingga 29%.
2	Li et al.	2024	Wireless Fingerprinting	Neighborhood Selection, Feature Scaling	Mengusulkan <i>Enhanced</i> WAKNN yang menyesuaikan bobot fitur berdasarkan geometri tetangga terdekat untuk mengurangi error pada estimasi lokasi.
3	Hapsari & Indriani	2020	Judul Forum Mahasiswa	Tokenisasi, Filtering, Stemming	Membandingkan performa NBC dan KNN, serta menyoroti keunggulan WAKNN dalam menangani fitur kata yang sangat banyak pada klasifikasi teks otomatis.
4	Taunk & De	2024	Berbagai Domain (Survey)	Data Cleaning, Feature Weighting	Melakukan tinjauan algoritma tetangga terdekat dan menyimpulkan bahwa WAKNN dengan optimasi <i>hill-climbing</i> sangat efektif untuk dataset dengan fitur redundan.
5	Yahya & Hidayanti	2020	Data Penjualan Vape	Label Encoding, Min-Max Scaling	Menggunakan prinsip pembobotan fitur pada KNN untuk mengklasifikasikan efektivitas penjualan, membuktikan bahwa fitur tertentu memiliki korelasi lebih tinggi terhadap hasil.

2. Data

Sumber data: <https://www.kaggle.com/datasets/nikhil7280/weather-type-classification>

No	Temperature	Humidity	Wind Speed	Precipitation (%)	Cloud Cover	Weather Type
1	14.0	73	9.5	82.0	partly cloudy	Rainy
2	39.0	96	8.5	71.0	partly cloudy	Cloudy
3	30.0	64	7.0	16.0	clear	Sunny
4	38.0	83	1.5	82.0	clear	Sunny
5	27.0	74	17.0	66.0	overcast	Rainy

Dataset ini dirancang untuk melatih model pembelajaran mesin dalam mengklasifikasikan tipe cuaca berdasarkan berbagai parameter meteorologi. Berikut adalah detail statistik dan karakteristik datanya:

2.1 Statistik Deskriptif (Fitur Numerik)

Dataset ini memiliki **13.200 baris data** tanpa ada nilai yang kosong (*missing values*), yang menunjukkan dataset ini sangat bersih dan siap diolah.

Fitur	Satuan	Karakteristik
Temperature	°C	Rentang sangat lebar, mencakup cuaca ekstrem dingin hingga sangat panas.
Humidity	%	Terdapat beberapa nilai di atas 100% yang mungkin merupakan <i>outlier</i> atau kondisi saturasi ekstrem.
Wind Speed	km/h	Menunjukkan variasi dari udara tenang hingga angin kencang.
Precipitation	%	Peluang terjadinya hujan/salju, rata-rata berada di angka sedang (53%).
Atm. Pressure	hPa	Tekanan atmosfer yang bervariasi tergantung lokasi (gunung/pesisir).
UV Index	Indeks	Mengukur intensitas radiasi matahari; angka 14 menunjukkan paparan sangat ekstrem.

Visibility	km	Jarak pandang rata-rata cukup rendah (5.4 km), dipengaruhi awan/kabut/salju.
-------------------	----	--

2.2 Distribusi Data Kategorikal

Fitur kategorikal memberikan konteks lingkungan dan waktu terhadap parameter numerik di atas.

- **Cloud Cover (Tutupan Awan):**
 - **Overcast (Mendung Total):** 6.090 data (Paling dominan).
 - **Partly Cloudy:** 4.560 data.
 - **Clear:** 2.139 data.
 - **Cloudy:** 411 data.
- **Season (Musim):**
 - Didominasi oleh musim **Winter** (5.610 data), diikuti oleh Spring, Autumn, dan Summer yang masing-masing berjumlah sekitar 2.500 data. Ini menunjukkan dataset mungkin sedikit condong ke kondisi cuaca dingin.
- **Location (Lokasi):**
 - Tersebar merata antara **Inland** (Pedalaman: 4.816) dan **Mountain** (Pegunungan: 4.813), dengan porsi lebih sedikit di **Coastal** (Pesisir: 3.571).

2.3 Distribusi Kelas (Target Variable)

Fitur Weather Type adalah target yang ingin diprediksi. Dataset ini bersifat Balanced Dataset (Seimbang), yang sangat bagus untuk algoritma WAKNN agar tidak bias terhadap kelas tertentu.

- **Rainy:** 3.300 data (25%)
- **Cloudy:** 3.300 data (25%)
- **Sunny:** 3.300 data (25%)
- **Snowy:** 3.300 data (25%)

3. Praproses

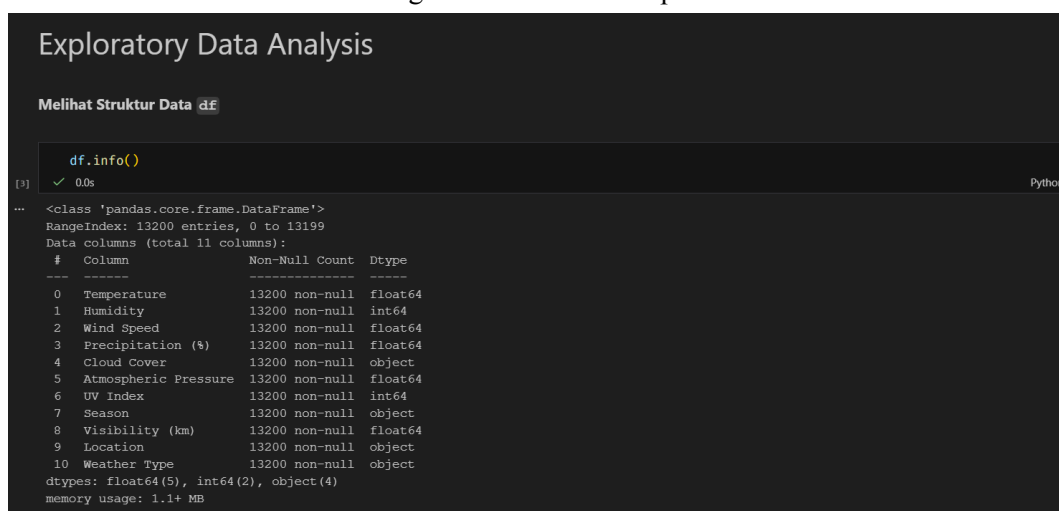
Jauh sebelum melakukan pra proses, terlebih dahulu kami melakukan proses yang namanya **Exploratory Data Analysis**. Menurut Website GeeksForGeeks, “*EDA is an important step in data science and data analytics as it visualizes data to understand its main features, find patterns and discover how different parts of the data are connected*”.

Maksudnya adalah, EDA merupakan salah satu proses yang sangat penting untuk dilakukan. Proses ini memungkinkan analis untuk memahami data secara mendalam, baik memahami bagaimana maksud data tersebut, apa fitur-fitur utamanya dari data tersebut, apakah datanya kotor atau sebagian sudah bersih. Apakah ada pola tertentu yang jika dilihat sekilas, tidak ada masalah, namun ketika di reset lebih dalam, ada hal yang janggal.

Setelah proses EDA dilakukan, di sana kami mendapatkan banyak sekali insight tentang datanya. Dari proses tersebut juga, kami mendapatkan insight tentang data apa yang perlu kami lakukan pada tahapan pra proses. Apakah kami perlu melakukan normalisasi atau tidak, apakah kami perlu seleksi fitur atau tidak, dan lain-lain.

Berikut adalah foto-foto dan penjelasan singkat dari setiap prosesnya.

Mengecek Column dan Tipe Data.



```
Exploratory Data Analysis

Melihat Struktur Data df

df.info()
[3] ✓ 0.0s Python

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 13200 entries, 0 to 13199
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   Temperature            13200 non-null  float64
1   Humidity               13200 non-null  int64  
2   Wind Speed             13200 non-null  float64
3   Precipitation (%)      13200 non-null  float64
4   Cloud Cover            13200 non-null  object  
5   Atmospheric Pressure   13200 non-null  float64
6   UV Index               13200 non-null  int64  
7   Season                 13200 non-null  object  
8   Visibility (km)        13200 non-null  float64
9   Location               13200 non-null  object  
10  Weather Type           13200 non-null  object  
dtypes: float64(5), int64(2), object(4)
memory usage: 1.1+ MB
```

Insight dari Code Sebelumnya.

Insight:

- Jumlah data untuk semua fitur, sama. Artinya tidak ada missing value
- Tipe data untuk setiap fitur sudah benar
- Jumlah baris sebanyak 13.200 dengan 11 kolom

Mengecek Deskriptif Statistik Data.

Cek Descriptive Statistic

```
df.describe()
```

✓ 0.0s Python

	Temperature	Humidity	Wind Speed	Precipitation (%)	Atmospheric Pressure	UV Index	Visibility (km)
count	13200.000000	13200.000000	13200.000000	13200.000000	13200.000000	13200.000000	13200.000000
mean	19.127576	68.710833	9.832197	53.644394	1005.827896	4.005758	5.462917
std	17.386327	20.194248	6.908704	31.946541	37.199589	3.856600	3.371499
min	-25.000000	20.000000	0.000000	0.000000	800.120000	0.000000	0.000000
25%	4.000000	57.000000	5.000000	19.000000	994.800000	1.000000	3.000000
50%	21.000000	70.000000	9.000000	58.000000	1007.650000	3.000000	5.000000
75%	31.000000	84.000000	13.500000	82.000000	1016.772500	7.000000	7.500000
max	109.000000	109.000000	48.500000	109.000000	1199.210000	14.000000	20.000000

Mengecek dan Memastikan Data Missing Value.

Cek Missing Value

```
df.isna().sum()
```

[5] ✓ 0.0s

Temperature	0
Humidity	0
Wind Speed	0
Precipitation (%)	0
Cloud Cover	0
Atmospheric Pressure	0
UV Index	0
Season	0
Visibility (km)	0
Location	0
Weather Type	0
dtype: int64	

Insight:

- Terbukti bahwa tidak ada data yang missing

Mengecek dan Memastikan Duplikasi Data.

Cek Duplikasi Data

```
print(f'Jumlah data duplikat: {df.duplicated().sum()} buah data')
```

✓ 0.0s Python

Jumlah data duplikat: 0 buah data

Melihat Distribusi Data per Fitur,

Univariate Analysis

```
# Pisahkan kolom numerik dan kategorikal
num_cols_univysis = ['Temperature', 'Humidity', 'Wind Speed', 'Precipitation (%)', 'Atmospheric Pressure', 'UV Index', 'Visibility']
cat_cols_univysis = ['Cloud Cover', 'Season', 'Location', 'Weather Type']
```

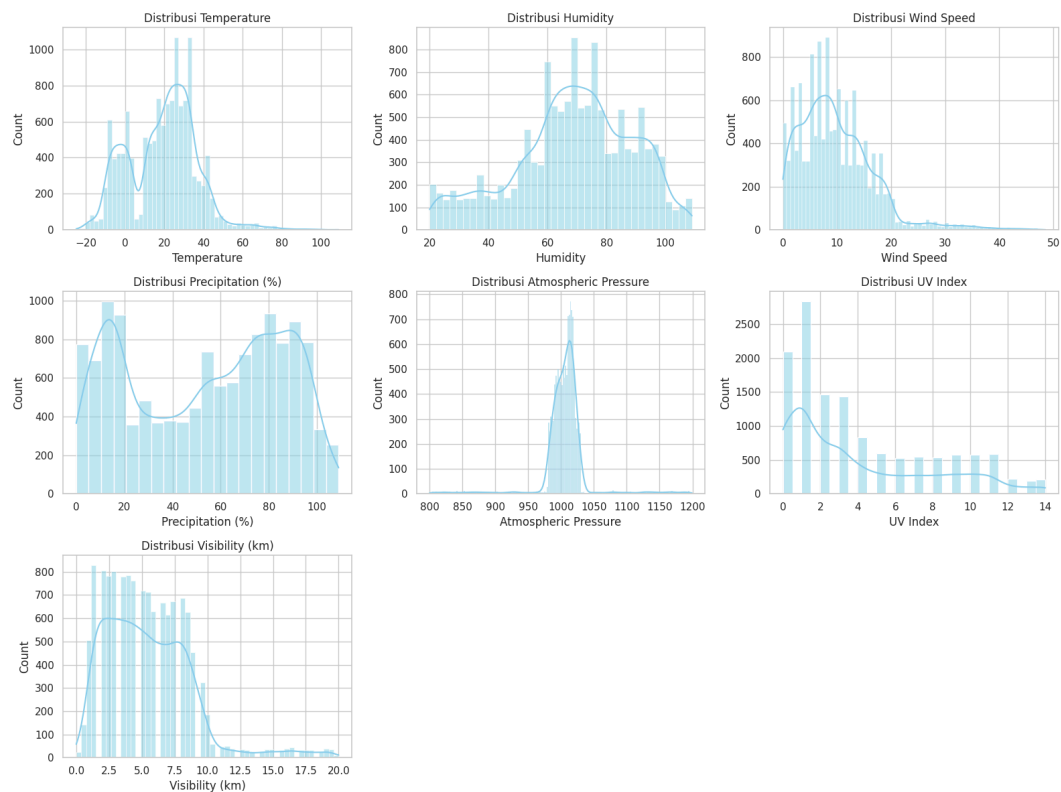
[7] ✓ 0.0s Python

```
# Set tema visualisasi
sns.set_theme(style='whitegrid')
```

✓ 0.0s Python

```
# Analisis Variabel Numerik - Melihat Distribusi Data
plt.figure(figsize=(16, 12))
for i, col in enumerate(num_cols_univsys, 1):
    plt.subplot(3, 3, i)
    sns.histplot(df[col], kde=True, color='skyblue')
    plt.title(f'Distribusi {col}')
plt.tight_layout()
plt.show()
```

✓ 1.6s Python



Insight dari Code Sebelumnya.

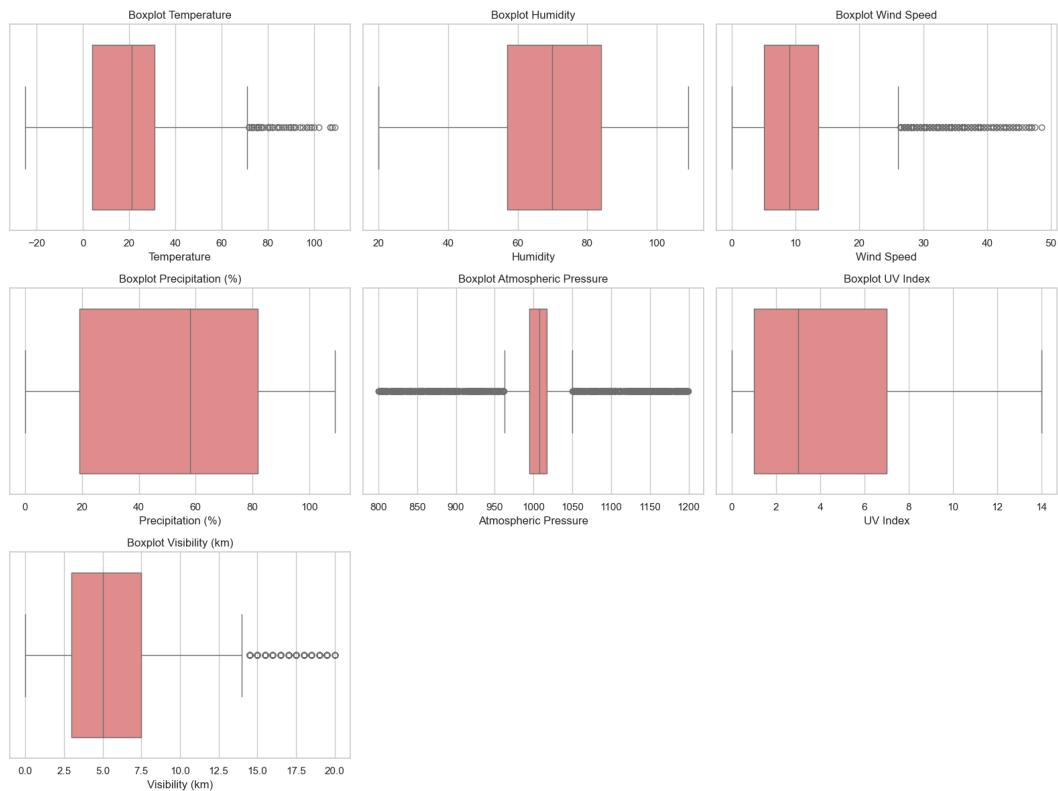
Insight:

- Pada histogram `Humidity`, terdapat beberapa data yang > 100%. Ini tidak masuk akal.
- Pada histogram `Temperature`, terdapat beberapa data suhu yang > 60 derajat. Ini sangat tidak wajar.
- Kedua hal itu harus dihapus pada proses **Preprocessing**

Mengecek Outlier dengan Boxplot.

```
# Analisis Outlier
plt.figure(figsize=(16, 12))
for i, col in enumerate(num_cols_univsys, 1):
    plt.subplot(3, 3, i)
    sns.boxplot(x=df[col], color='lightcoral')
    plt.title(f'Boxplot {col}')
plt.tight_layout()
plt.show()
```

✓ 0.6s Python



Insight dari Code Sebelumnya.

Insight:

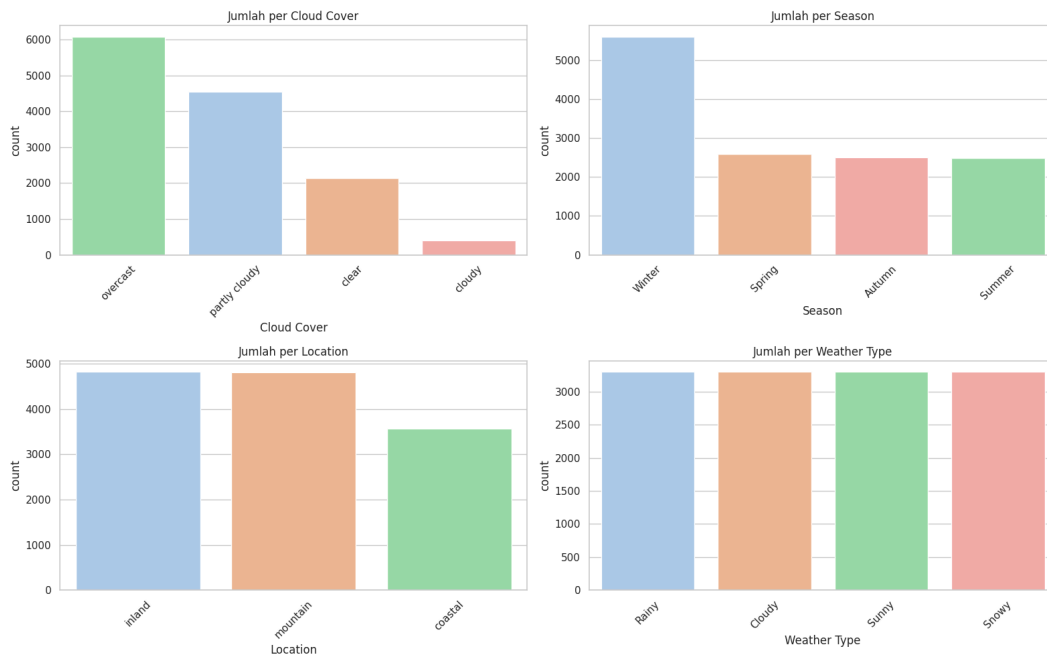
- Pada fitur `Temperature`, `Wind Speed`, dan `Atmospheric Pressure` memiliki banyak sekali outlier. Ini perlu ditangani, karena algoritma `WAKNN` sensitif terhadap outlier
- Untuk fitur `Visibility` juga memiliki outlier, namun tidak se-ekstrem 3 fitur sebelumnya
- Sedangkan sisanya, memiliki kondisi data yang baik

Cek Distribusi Kemunculan Data Pada Beberapa Fitur.

```
# Analisis Variabel Kategorikal - Melihat Frekuensi Kemunculan
plt.figure(figsize=(16, 10))
for i, col in enumerate(cat_cols_univysis, 1):
    plt.subplot(2, 2, i)
    order = df[col].value_counts().index
    sns.countplot(data=df, x=col, hue=col, palette='pastel', order=order, legend=False)
    plt.title(f'Jumlah per {col}')
    plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

✓ 0.6s

Python



Insight dari Code Sebelumnya.

Insight:

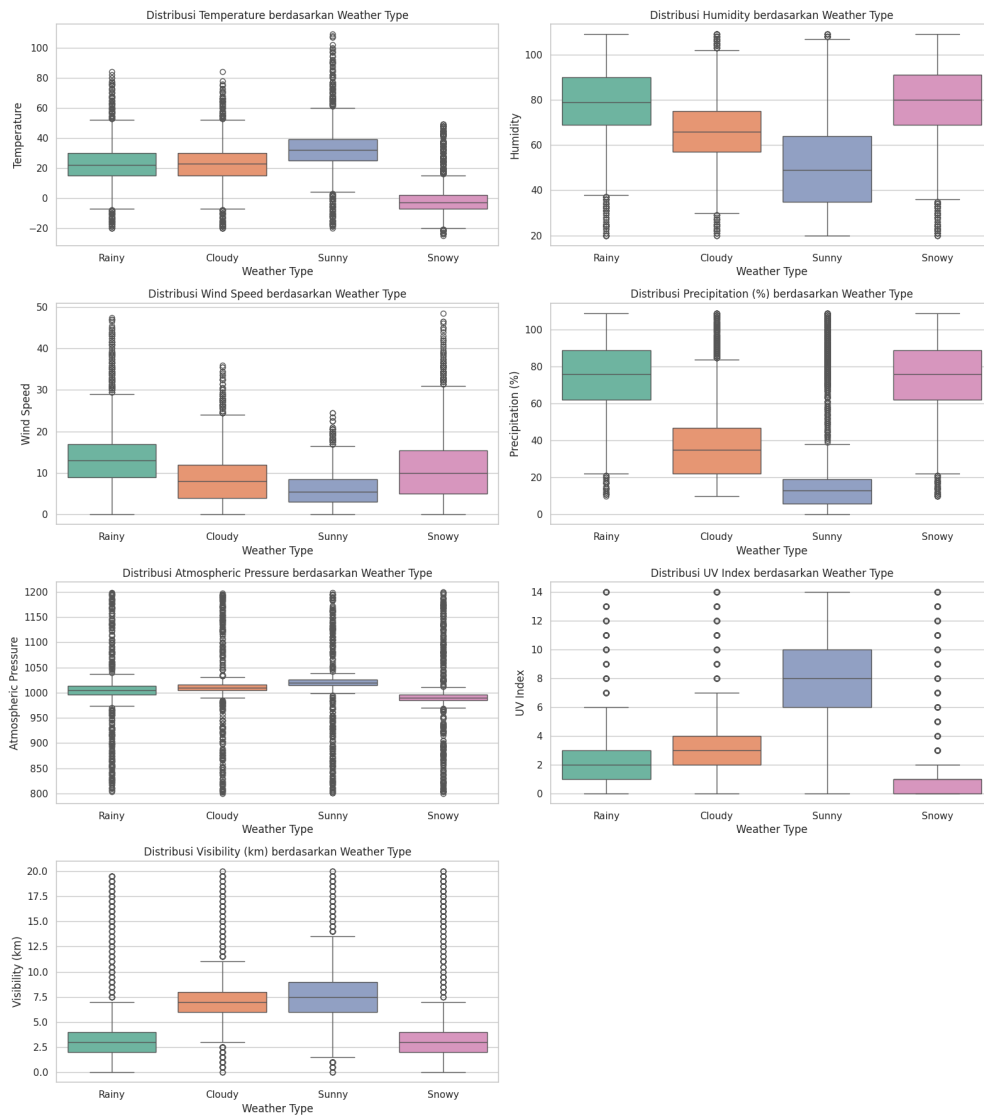
- Dari sini terlihat bahwa untuk label atau fitur `Weather Type`, jumlah valuenya nya balance

Melihat Korelasi Antar Fitur dengan Fitur Label.

Bivariate & Multivariate Analysis

```
# Mengatur kolom
cat_cols_bivysis = ['Cloud Cover', 'Season', 'Location']
target = 'Weather Type'

# Bivariate: Numerik vs Target - Melihat Fitur mana yang Membedakan Tiap Cuaca
plt.figure(figsize=(16, 18))
for i, col in enumerate(num_cols_univysis, 1):
    plt.subplot(4, 2, i)
    sns.boxplot(data=df, x=target, y=col, hue=target, palette='Set2')
    plt.title(f'Distribusi {col} berdasarkan {target}')
plt.tight_layout()
plt.show()
```



Insight dari Code Sebelumnya.

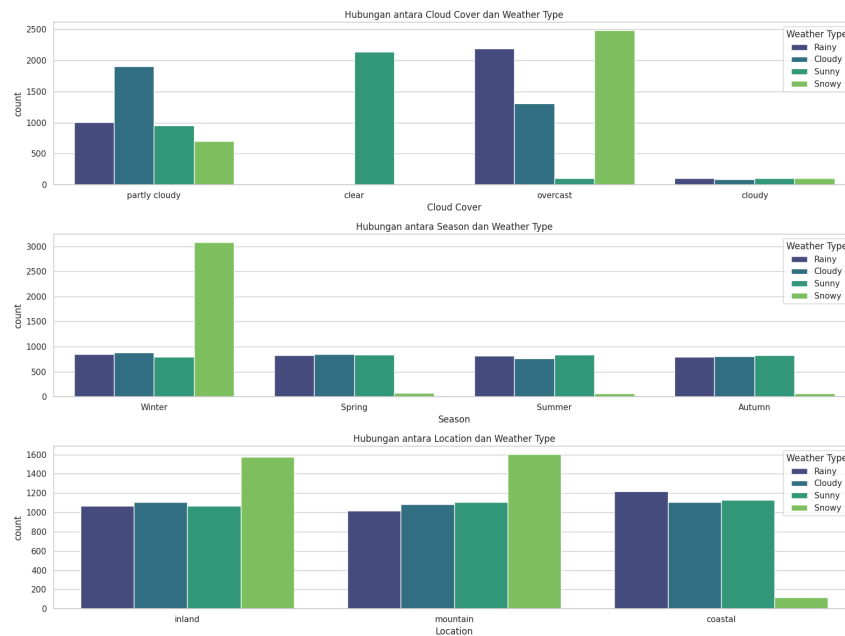
Insight:

- Dari sini terlihat bahwa `weather_type` sangat dipengaruhi oleh beberapa fitur

Melihat Korelasi Lokasi dengan Weather Type.

```
# Bivariate: Kategorikal vs Target - Melihat apakah lokasi atau musim didominasi cuaca tertentu
plt.figure(figsize=(16, 12))
for i, col in enumerate(cat_cols_bivysis, 1):
    plt.subplot(3, 1, i)
    sns.countplot(data=df, x=col, hue=target, palette='viridis')
    plt.title(f'Hubungan antara {col} dan {target}')
    plt.legend(title='Weather Type')
plt.tight_layout()
plt.show()
```

✓ 0.7s Python



Insight dari Code Sebelumnya.

Insight:

- Di `coastal` atau `wilayah pesisir`, cuaca `snowy` sangat jarang terjadi
- Pada musim `Winter`, cuaca `Snowy` justru yang sangat sering terjadi

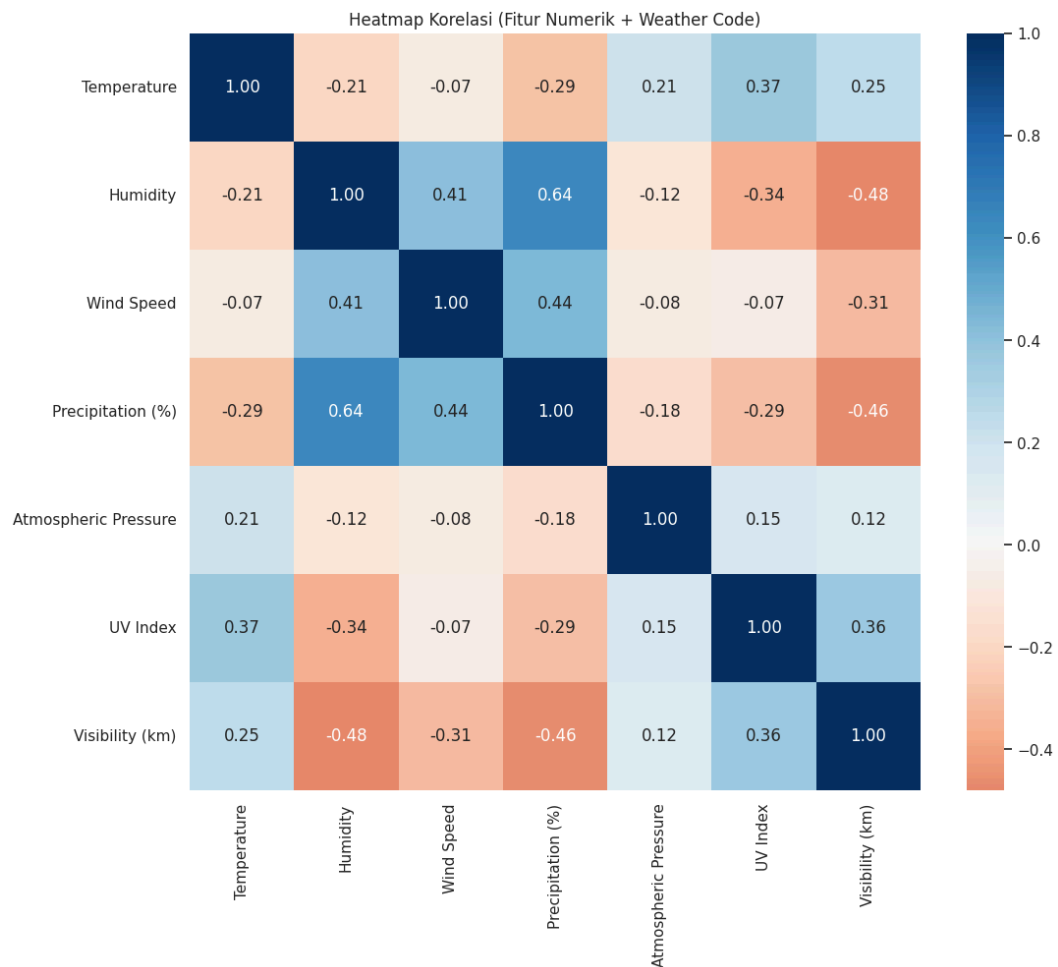
Melihat Korelasi antar Fitur dengan Heatmap.

```
# Multivariate: Heatmap Korelasi - Melihat korelasi antar fitur
# Konversi target ke kode angka sementara
df_encoded = df.copy()
df_encoded['Weather_Code'] = df_encoded[target].astype('category').cat.codes
numeric_df = df_encoded.select_dtypes(include=['float64', 'int64'])

plt.figure(figsize=(12, 10))
sns.heatmap(numeric_df.corr(), annot=True, cmap='RdBu', fmt='.2f', center=0)
plt.title('Heatmap Korelasi (Fitur Numerik + Weather Code)')
plt.show()
```

✓ 0.0s Python

✓ 0.2s Python



Insight dari Code Sebelumnya.

Insight:

- Terlihat bahwa antara fitur `Wind Speed` dengan `Precipitation` memiliki korelasi fitur yang cukup tinggi, yaitu sebesar **0,64**
- Sedangkan untuk korelasi fitur `Visibility` dengan `Wind Speed`, `Visibility` dengan `Precipitation` maupun sebaliknya, memiliki korelasi fitur yang rendah, bahkan hingga **menyentuh angka minus**

Tahapan EDA.

Preprocessing Data

Menghapus Outlier

```
# Hapus outlier Humidity yang > 100 dan Temperature yang > 60
df_cleaned = df[(df['Humidity'] ≤ 100) & (df['Temperature'] ≤ 60)].copy()
```

✓ 0.0s

Python

Pada proses kali ini, sesuai insight saat mengecek distribusi data. Terdapat beberapa data di fitur Humidity dan Temperature yang nilainya tidak normal. Maka pada proses kali ini, difilter datanya.

Melakukan One-Hot Encoding untuk Fitur Nominal.

```
Encoding Variabel Kategorikal

# One-Hot Encoding untuk fitur Cloud Cover
nominal_cols = ['Cloud Cover', 'Season', 'Location']
df_preprocessed = pd.get_dummies(df_cleaned, columns=nominal_cols, dtype=int)

# Label Encoding untuk fitur Weather Type
label_encoder = LabelEncoder()
df_preprocessed['Weather Type'] = label_encoder.fit_transform(df_preprocessed['Weather Type'])
```

Melakukan Label Encoding untuk data Ordinal Serta Hasil Encodingnya.

```
# Melihat hasil encoding
for i, label in enumerate(label_encoder.classes_):
    print(f'Nilai hasil encoding {i}: {label}')

Nilai hasil encoding 0: Cloudy
Nilai hasil encoding 1: Rainy
Nilai hasil encoding 2: Snowy
Nilai hasil encoding 3: Sunny
```


4. Analisis metode

No	Temperature	Humidity	Wind Speed	Precipitation (%)	Atmospheric Pressure	UV Index	Visibility (km)	Cloud Cover_clear	Cloud Cover_cloudy	Cloud Cover_overcast	Cloud Cover_partlycloudy	Season_autumn	Season_spring	Season_summer	Season_winter	Location_coastal	Location_inland	Location_mountain	Keterangan
1	0,70	0,31	0,12	0,79	0,20	0,14	0,05	0	0	0	1	0	1	0	0	0	0	1	Cloudy
2	0,49	0,88	0,23	0,90	0,54	0,21	0,20	0	0	1	0	0	0	0	1	0	1	0	Rainy
3	0,76	0,58	0,16	0,46	0,55	0,36	0,28	1	0	0	0	1	0	0	0	0	0	1	Sunny
4	0,62	0,56	0,28	0,44	0,55	0,29	0,28	0	0	0	1	0	0	1	0	0	0	1	Cloudy
5	0,63	0,78	0,37	0,72	0,48	0,21	0,20	0	0	1	0	0	1	0	0	0	1	0	Rainy
6	0,69	0,94	0,32	0,83	0,54	0,79	0,58	1,00	0,00	0,00	0,00	0,00	1,00	0,00	0,00	1,00	0,00	0,00	Sunny

Disini yang harus diceritakan

1. Parameter yang akan digunakan
2. Pelatihan menggunakan 5 data latih diatas
3. Hasil pelatihan
4. Pengujiannya

5. Hasil Implementasi

Disini kalian cerita tentang hasil akurasi, f1-score, recall, dan precision menggunakan:

1. Hasil seleksi fitur berapakah yang akurasinya terbaik dan tanpa seleksi fitur.
2. Fitur apa saja yang digunakan untuk menghasilkan performa yang terbaik.
3. Performa klasifikasi jika sebelumnya dilakukan pengurangan dimensi pada fitur.
4. Bandingkan dengan K-NN standar

6. Source code

7. Daftar Pustaka

- [1] S. Zheng, Y. Wang, and J. Li, "Weighted K-NN Classification Method of Bearings Fault Diagnosis With Multi-Dimensional Sensitive Features," *IEEE Access*, vol. 9, pp. 41243–41253, Mar. 2021, doi: 10.1109/ACCESS.2021.3065094.
- [2] L. Zhang, X. Li, and Y. Wang, "Partial Weighted K-nearest Neighbor Classification of Incomplete Data," in *Proc. 2022 IEEE 4th International Conference on Civil Aviation Safety and Information Technology (ICCASIT)*, Oct. 2022, pp. 638–642, doi: 10.1109/ICCASIT55261.2022.10013621.
- [3] M. Taunk and S. De, "Performance analysis of adaptive K for weighted K-nearest neighbor based indoor positioning," in *Proc. 2022 IEEE 95th Vehicular Technology Conference (VTC2022-Spring)*, Jun. 2022, pp. 1–5, doi: 10.1109/VTC2022-Spring54377.2022.9860699.
- [4] Z. Han and L. Zhao, "Compactness-Weighted KNN Classification Algorithm," *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 15, no. 9, Sep. 2024. [Online]. Available: <https://thesai.org/Publications/IJACSA>.
- [5] A. Nababan, J. Sembiring, and E. Nababan, "Improving the Accuracy of Features Weighted k-Nearest Neighbor using Distance Weight," in *Proc. 2nd International Conference on Computing and Informatics Engineering (IC2IE)*, 2019, pp. 1-6.
- [6] N. Aggarwal, "What is Exploratory Data Analysis?," *GeeksforGeeks*, Oct. 2025. [Online]. Available: <https://www.geeksforgeeks.org/data-analysis/what-is-exploratory-data-analysis/>.